

1. Descripción General del Sistema

El sistema **Banca BP** se define como una plataforma de servicios financieros digitales de nueva generación, diseñada para ofrecer una experiencia de usuario fluida, segura y altamente disponible. La plataforma responde a la necesidad de la entidad financiera BP de modernizar su ecosistema, permitiendo a los clientes realizar operaciones críticas como la consulta de movimientos, la ejecución de transferencias interbancarias y propias, así como un proceso de *onboarding* digital con verificación biométrica en tiempo real.

Arquitectura de Referencia: La solución se fundamenta en una arquitectura de **microservicios desacoplados** sobre **Kubernetes (AWS EKS)**, garantizando escalabilidad horizontal y resiliencia. El diseño sigue el **modelo C4** para asegurar la trazabilidad desde la visión de negocio hasta la implementación técnica.

Pilares del Sistema:

- **Conectividad y Seguridad Perimetral:** Utiliza **Kong** como *API Gateway* de alto rendimiento para la centralización de políticas de seguridad (Rate Limiting, WAF) y enrutamiento inteligente.
- **Gestión de Identidad (IdP):** La identidad y acceso se delegan en **Keycloak**, implementando estándares **OAuth2/OIDC** para asegurar el acceso basado en roles y flujos de autenticación robustos.
- **Seguridad de Secretos:** Integración nativa con **HashiCorp Vault** para la gestión dinámica de credenciales, minimizando el riesgo de exposición de información sensible mediante credenciales efímeras (TTL bajo).
- **Consistencia y Comunicación:** La comunicación asíncrona está soportada por un *message broker* **RabbitMQ** en cluster, implementando el patrón **Saga Orquestado** para asegurar la consistencia eventual en transacciones distribuidas.
- **Persistencia y Auditoría:** Se emplea una estrategia de persistencia polígota combinando bases de datos relacionales (Aurora) para el estado del negocio y almacenamiento no relacional (DynamoDB) para la auditoría inmutable mediante *Event Sourcing*.

El sistema no solo cumple con los requisitos funcionales de la banca digital actual, sino que se alinea con las mejores prácticas del **AWS Well-Architected Framework**, priorizando la observabilidad.

Objetivos Estratégicos

- Incrementar la adopción de canales digitales.
- Reducir tiempos de atención y procesamiento de operaciones financieras.
- Mejorar la experiencia del cliente mediante procesos digitales.
- Garantizar cumplimiento regulatorio y auditoría completa.
- Asegurar continuidad operativa y alta disponibilidad.

Capacidades de Negocio

Capacidad	Descripción
Gestión de Clientes	Consulta y administración de información del cliente
Transferencias	Transferencias internas e interbancarias
Onboarding Digital	Vinculación digital con biometría
Gestión de Seguridad	Control de acceso y autenticación
Gestión de Auditoría	Registro y trazabilidad de eventos

2. Decisiones Arquitectónicas con Justificación

Esta sección detalla las elecciones tecnológicas críticas que definen la robustez, seguridad y mantenibilidad de la plataforma **Banca BP**.

2.1 Orquestación: Kubernetes (AWS EKS)

- **Decisión:** Uso de AWS EKS para el despliegue de microservicios mediante contenedores.
- **Justificación Teórica 1: Portabilidad y Estandarización:** Kubernetes ofrece una abstracción del proveedor de nube que permite evitar el *vendor lock-in*. Su ecosistema (Helm, ArgoCD) facilita la implementación de prácticas GitOps.
- **Justificación Teórica 2: Gestión de Recursos y Auto-healing:** A diferencia de enfoques PaaS (como AWS Elastic Beanstalk), EKS permite un control granular de *sidecars* (ej. para el agente de Vault o proxies de mTLS) y reinicio automático de pods ante fallos de salud.
- **Alternativas Evaluadas:**
 - *AWS ECS Fargate:* Descartado por ofrecer menor flexibilidad en la configuración de redes y menor madurez en el ecosistema de orquestación de microservicios complejos.
 - *Serverless (AWS Lambda):* Descartado debido a los problemas de *cold starts* en aplicaciones Java Spring Boot y la complejidad de gestionar transacciones distribuidas de larga duración con Saga.

2.2 Patrón de Integración y Consistencia: Saga Orquestado (RabbitMQ)

- **Decisión:** Implementación de Saga Orquestado mediante RabbitMQ con colas de alta disponibilidad (*Quorum Queues*).

- **Justificación Teórica 1:** *Consistencia Eventual en Sistemas Distribuidos:* Al no poder utilizar transacciones ACID a través de múltiples microservicios (debido al Teorema CAP), la Saga permite mantener la consistencia del negocio mediante pasos compensatorios.
- **Justificación Teórica 2:** *Desacoplamiento Temporal:* RabbitMQ permite que el *Svc Transferencias* (orquestador) continúe operando incluso si los servicios de auditoría o notificaciones tienen picos de carga o latencia momentánea.
- **Alternativas Evaluadas:**
 - *Saga Coreografiada:* Descartado para el core bancario porque genera un grafo de dependencias difícil de auditar y rastrear ante fallos; el orquestador centralizado es más fácil de mantener.
 - *Two-Phase Commit (2PC):* Descartado por ser un patrón bloqueante (síncrono) que afecta drásticamente la disponibilidad y el rendimiento del sistema ante fallos de red.

2.3 Gestión de Identidad y Seguridad: Keycloak + Vault

- **Decisión:** Keycloak para manejo de tokens (OIDC) y HashiCorp Vault para secretos dinámicos.
- **Justificación Teórica 1:** *Separación de Preocupaciones (Separation of Concerns):* Delegar la identidad a Keycloak centraliza la auditoría de accesos y la gestión de ciclos de vida de tokens, manteniendo los microservicios agnósticos a la lógica de autenticación.
- **Justificación Teórica 2:** *Seguridad de Credenciales (Principle of Least Privilege):* El uso de motores de secretos dinámicos en Vault garantiza que las credenciales de DB tengan un TTL (Time-To-Live) breve, eliminando la existencia de contraseñas de larga duración que pudieran ser comprometidas.
- **Alternativas Evaluadas:**
 - *AWS Cognito:* Descartado por requerir integraciones más propietarias y limitaciones en la personalización de *flows* complejos requeridos en banca (ej. biometría específica).
 - *Gestión de secretos en Environment Variables:* Descartado por ser inseguro (vulnerabilidad de exposición en logs y procesos del sistema).

2.4 Persistencia y Auditoría: CQRS con DynamoDB

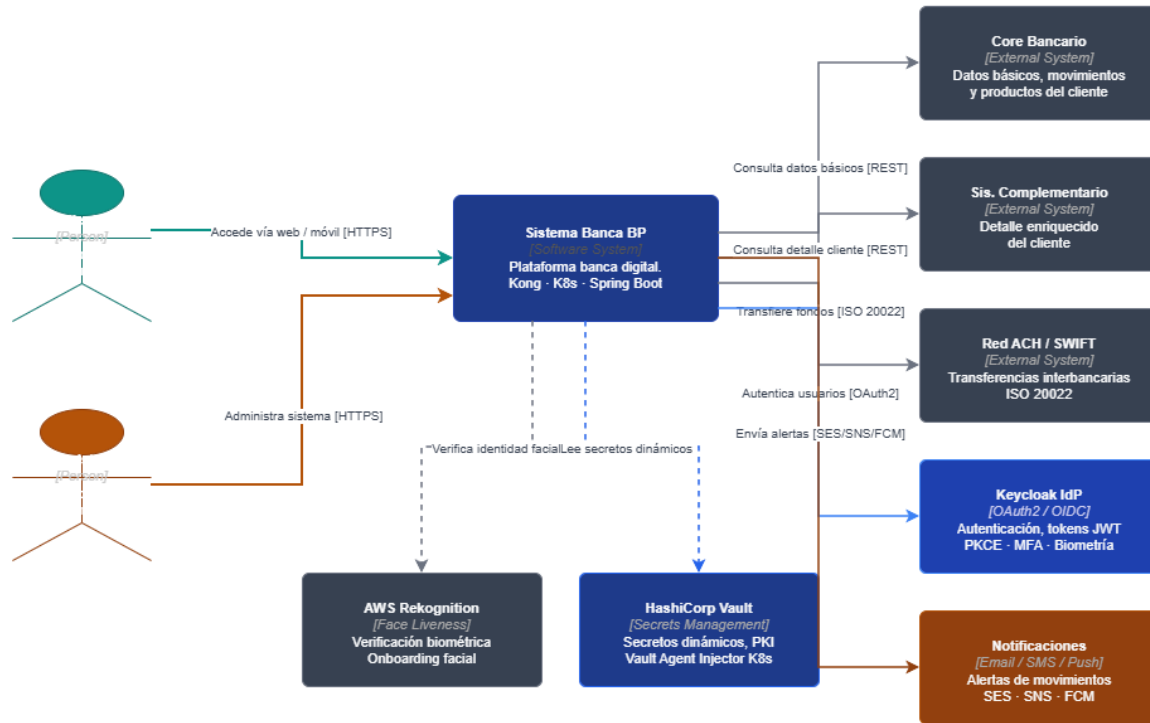
- **Decisión:** CQRS (*Command Query Responsibility Segregation*) usando Aurora para transaccionalidad y DynamoDB para eventos.
- **Justificación Teórica 1:** *Escalabilidad de Lectura/Escritura:* CQRS permite optimizar la carga de trabajo separando el modelo de escritura (transaccional) del modelo de lectura (optimizada para consulta rápida).
- **Justificación Teórica 2:** *Trazabilidad Inmutable:* DynamoDB, mediante *Streams*, actúa como una fuente de verdad histórica (Event Sourcing) para cualquier auditoría regulatoria, cumpliendo con la exigencia de inmutabilidad de los movimientos financieros.

- **Alternativas Evaluadas:**

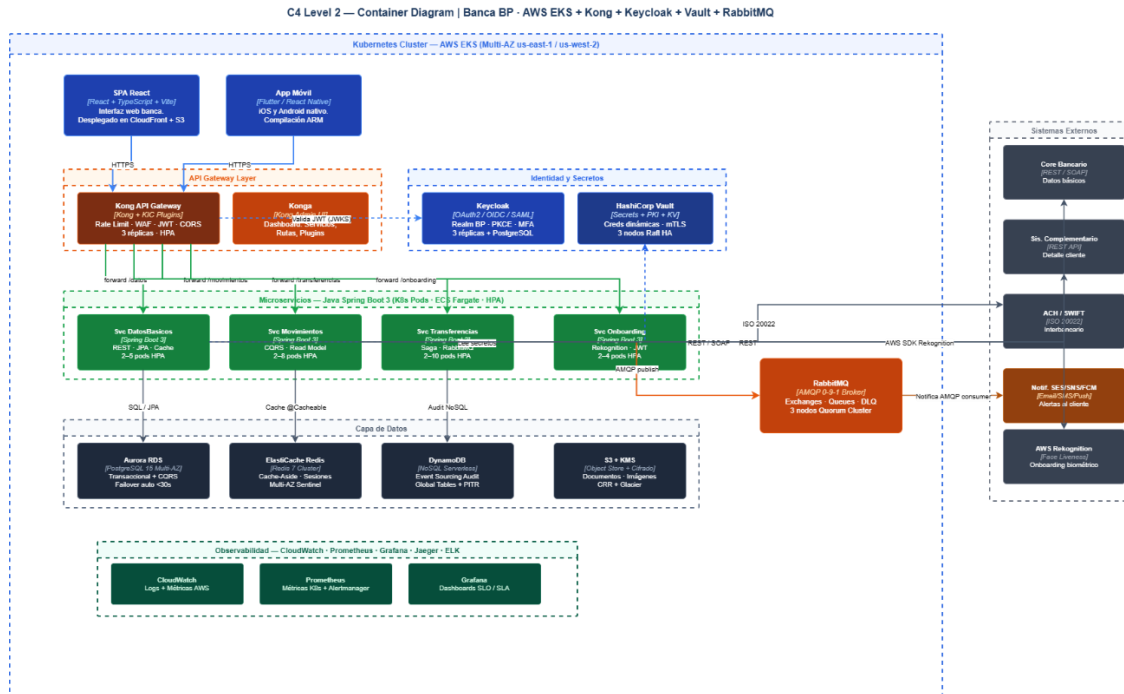
- *Base de datos única relacional:* Descartado porque la carga de auditoría masiva degradaría el rendimiento de las consultas transaccionales de saldo en tiempo real.
- *Logs basados en archivos (Flat files):* Descartado por carecer de capacidades de consulta compleja (ej. buscar por clientId + timestamp a gran escala).

3. C4 Nivel 1 — Diagrama de Contexto

C4 Level 1 — Diagrama de Contexto | Banca BP



4. C4 Nivel 2 — Diagrama de Contenedores



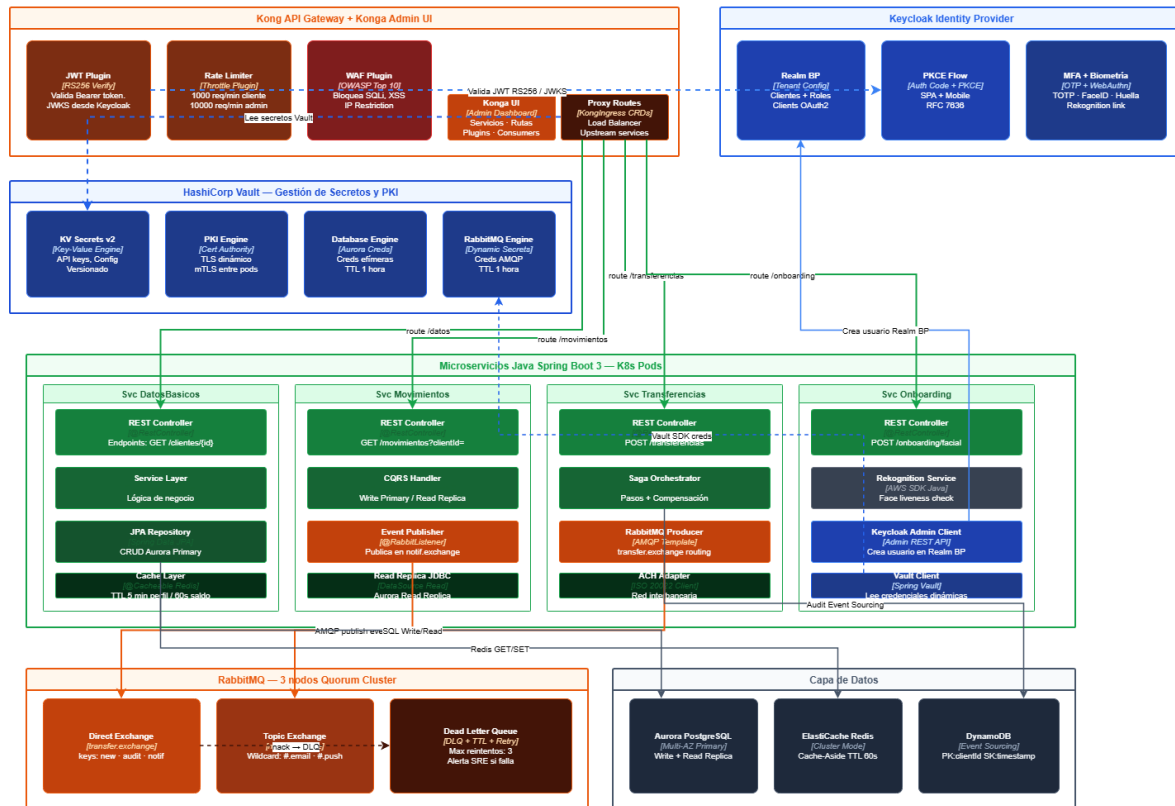
Front-end y Acceso: La estrategia combina SPAs (React) y aplicaciones multiplataforma (Flutter), protegidas por un API Gateway (Kong) que actúa como punto central de entrada, garantizando seguridad perimetral y limitación de tráfico (rate limiting).

Identidad y Seguridad: Se implementa un modelo Zero Trust centralizado. La autenticación es gestionada por Keycloak mediante estándares OIDC, mientras que la gestión de credenciales sensibles y certificados mTLS se delega a HashiCorp Vault, asegurando que el acceso a datos esté siempre cifrado y auditado.

Core de Negocio y Persistencia: El sistema utiliza microservicios en Java Spring Boot para la lógica de negocio, apoyándose en RabbitMQ para orquestar flujos de trabajo asíncronos mediante el patrón Saga, lo cual garantiza la integridad de las transferencias. Finalmente, la persistencia utiliza un enfoque híbrido: Aurora PostgreSQL para datos transaccionales (consistencia fuerte) y DynamoDB para auditoría (inmutabilidad mediante Event Sourcing), asegurando trazabilidad total frente a los requerimientos normativos mencionados en el ejercicio.

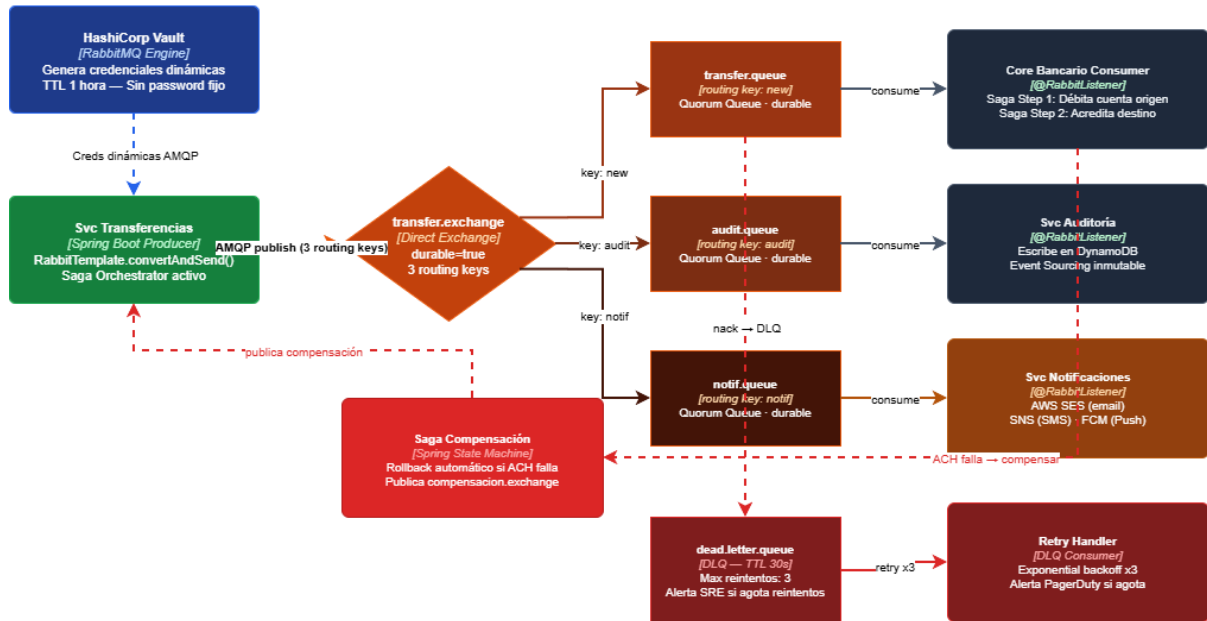
5. C4 Nivel 3 — Diagrama de Componentes

C4 Level 3 — Component Diagram | Kong · Keycloak · Vault · Spring Boot 3 · RabbitMQ



6. Diagrama de Flujo RabbitMQ — Patrón Saga Transferencias

Flujo RabbitMQ — Patrón Saga para Transferencias Interbancarias



Patrón: Saga Orquestado con compensación automática | Quorum Queues: replicación en 3 nodos | TTL mensaje: 30s | DLQ reintentos: 3x exponential backoff | Vault: creds dinámicas TTL 1h

7. Arquitectura de Autenticación con Keycloak + PKCE Flujo recomendado: Authorization Code + PKCE (RFC 7636) — obligatorio para SPA y apps móviles. Keycloak actúa como Authorization Server. Kong valida el JWT (RS256) en cada request sin consultar Keycloak, usando la clave pública expuesta en /.well-known/jwks.json.

8. Arquitectura de Negocio

Actores

- Cliente
- Administrador Plataforma
- Core Bancario
- Sistema Complementario
- ACH/SWIFT
- Plataforma de Notificaciones

9. Procesos Críticos

Consulta de Información

1. Cliente inicia consulta.
2. Sistema valida identidad.
3. Obtiene información desde sistemas internos.
4. Presenta información consolidada.

Transferencias

1. Cliente solicita transferencia.
2. Sistema valida autenticación.
3. Orquestador Saga inicia proceso.
4. ACH procesa transferencia.
5. Auditoría registra evento.
6. Cliente recibe notificación.

Onboarding

1. Cliente inicia vinculación.
2. Se captura biometría facial.
3. AWS Rekognition valida identidad.
4. Se crea usuario en Keycloak.
5. Se registra auditoría.

10. Arquitectura de Aplicación

Principios

- Arquitectura basada en microservicios.
- Desacoplamiento funcional.
- APIs REST.
- Comunicación asíncrona mediante eventos.
- Seguridad Zero Trust.

11. Microservicios

Svc Datos Básicos

Responsable de consultas de información del cliente.

Svc Movimientos

Responsable de movimientos financieros.

Svc Transferencias

Responsable de transferencias y orquestación Saga.

Svc Onboarding

Responsable del registro digital y validación biométrica.

12. Atributos de Calidad

Disponibilidad 99.95%

Escalabilidad Escalamiento horizontal mediante HPA.

Seguridad Autenticación centralizada y gestión dinámica de secretos.

Resiliencia Patrón Saga con compensación automática.

Trazabilidad Event Sourcing sobre DynamoDB.